

Operating Systems



⌘ An Operating System controls (manages) hardware and software.

- ☐ provides support for peripherals such as keyboard, mouse, screen, disk drives, ...

- ☐ software applications use the OS to communicate with peripherals.

- ☐ The OS typically manages (starts, stops, pauses, etc) applications.

Understanding Operating Systems

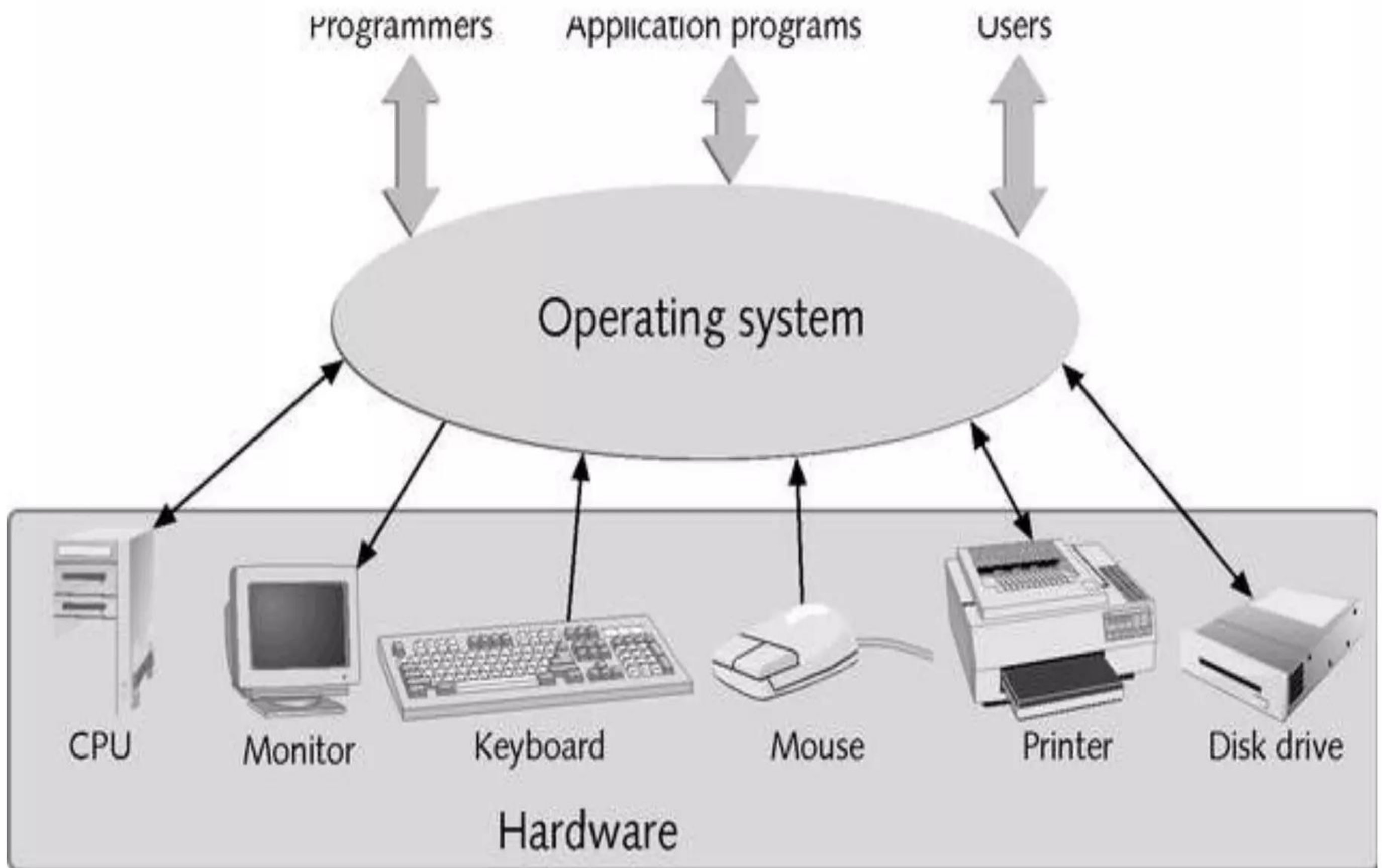


Figure 1-1 Operating system model

- **What Is UNIX?**

⌘ **UNIX** is a computer operating system, a control program that works with users to

- ⊠ run programs,

- ⊠ manage resources, and

- ⊠ communicate with other computer systems.

⌘ Several people can use a UNIX computer at the same time; hence UNIX is called a **multiuser** system. Any of these users can also run multiple programs at the same time; hence UNIX is called **multitasking**.

Introduction to Unix



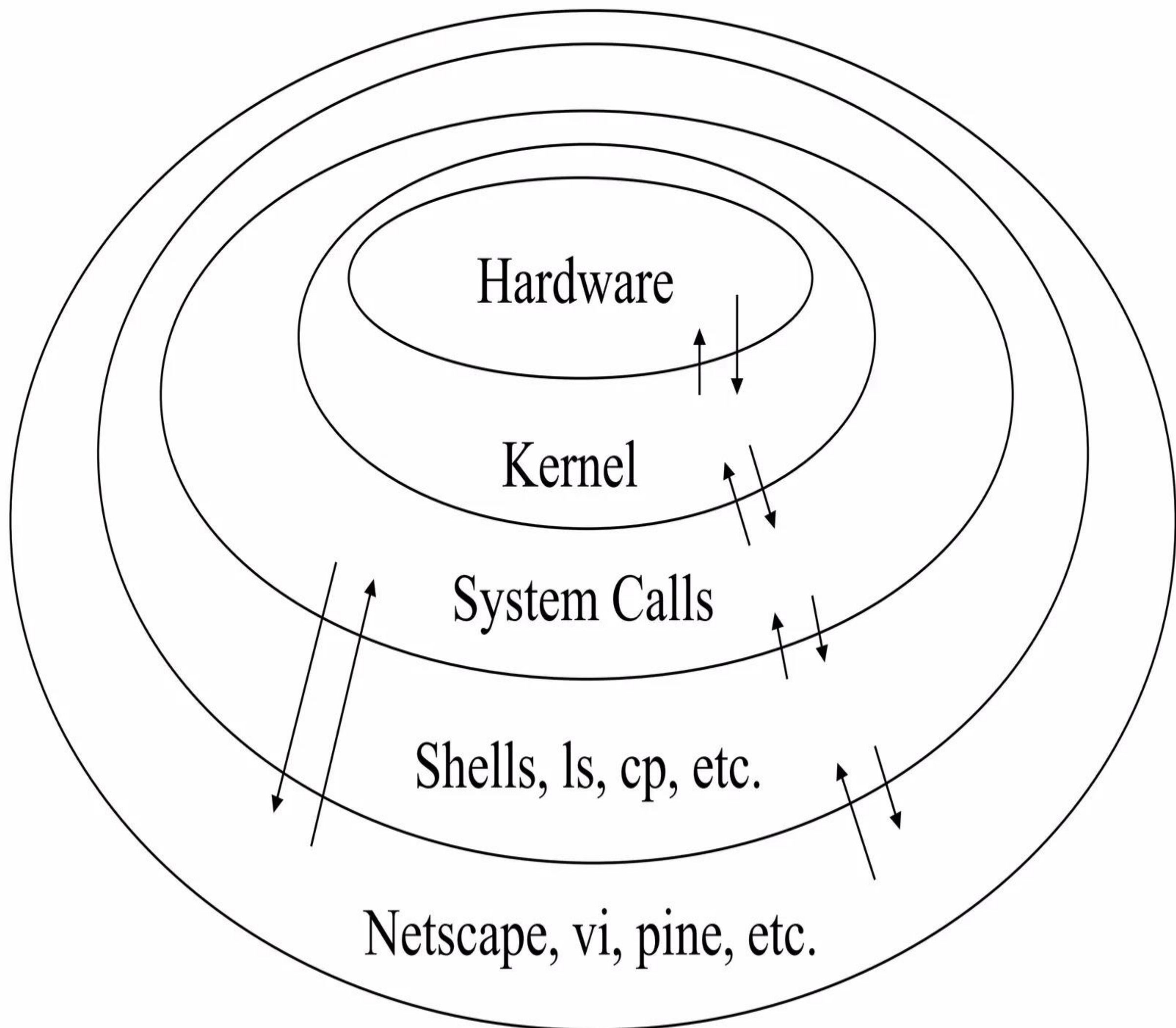
- ⌘ Unix was born in 1969 at Bell Laboratories, a research subdivision of American Telephone and Telegraph Company.
- ⌘ Some of the functions of the Unix operating system are:
 - ☑ it provides a filing system (write, copy, rename, move files)
 - ☑ it provides for the loading and executing of the user programs
 - ☑ it provides a communication link between the computer and its accessories (input-output devices as terminals, printers, disks, etc).

UNIX



- ⌘ Hardware is surrounded by the operating system software
- ⌘ Operating system is called the system kernel
 - ☑ The kernel is the core of an operating system and manages the machine's hardware resources (including the processor and the memory), and provides and controls the way any other software component can access these resources.
 - ☒ The kernel runs with a higher privilege than other programs.
- ⌘ Comes with a number of user services and interfaces
 - ☑ Shell
 - ☑ Components of the C compiler
 - ☑ GUI or Command Line Interface

Layers of a UNIX System



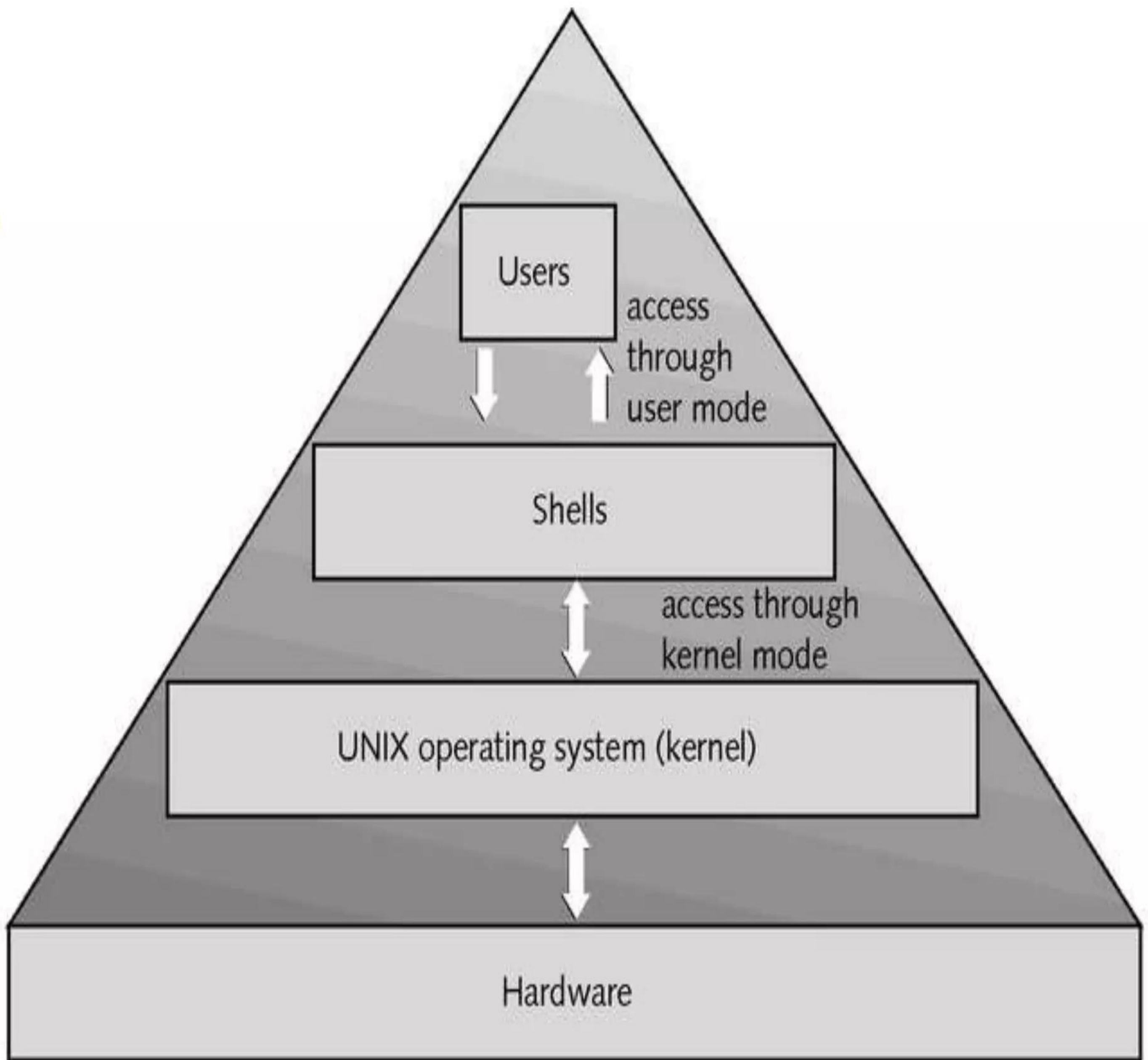
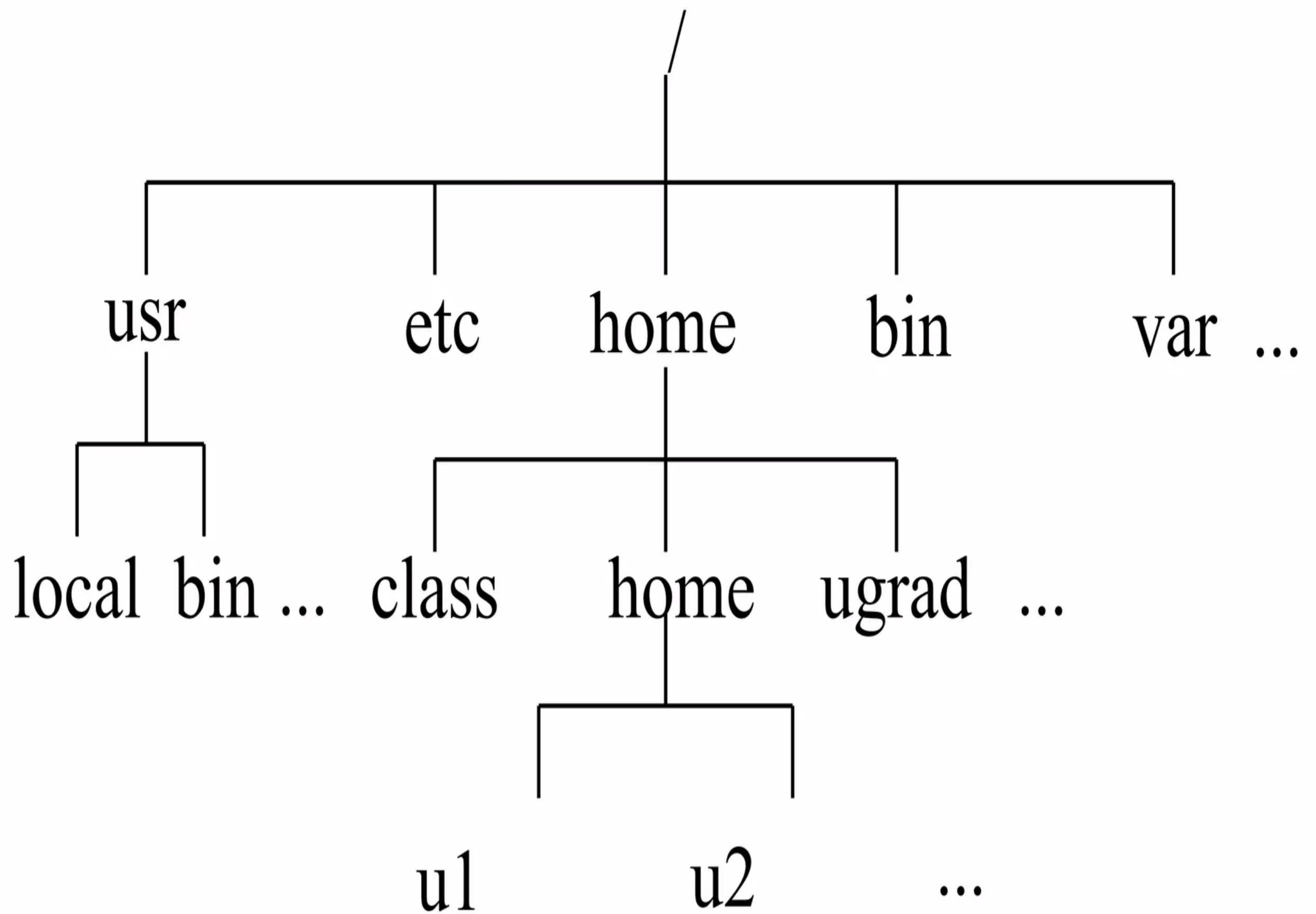
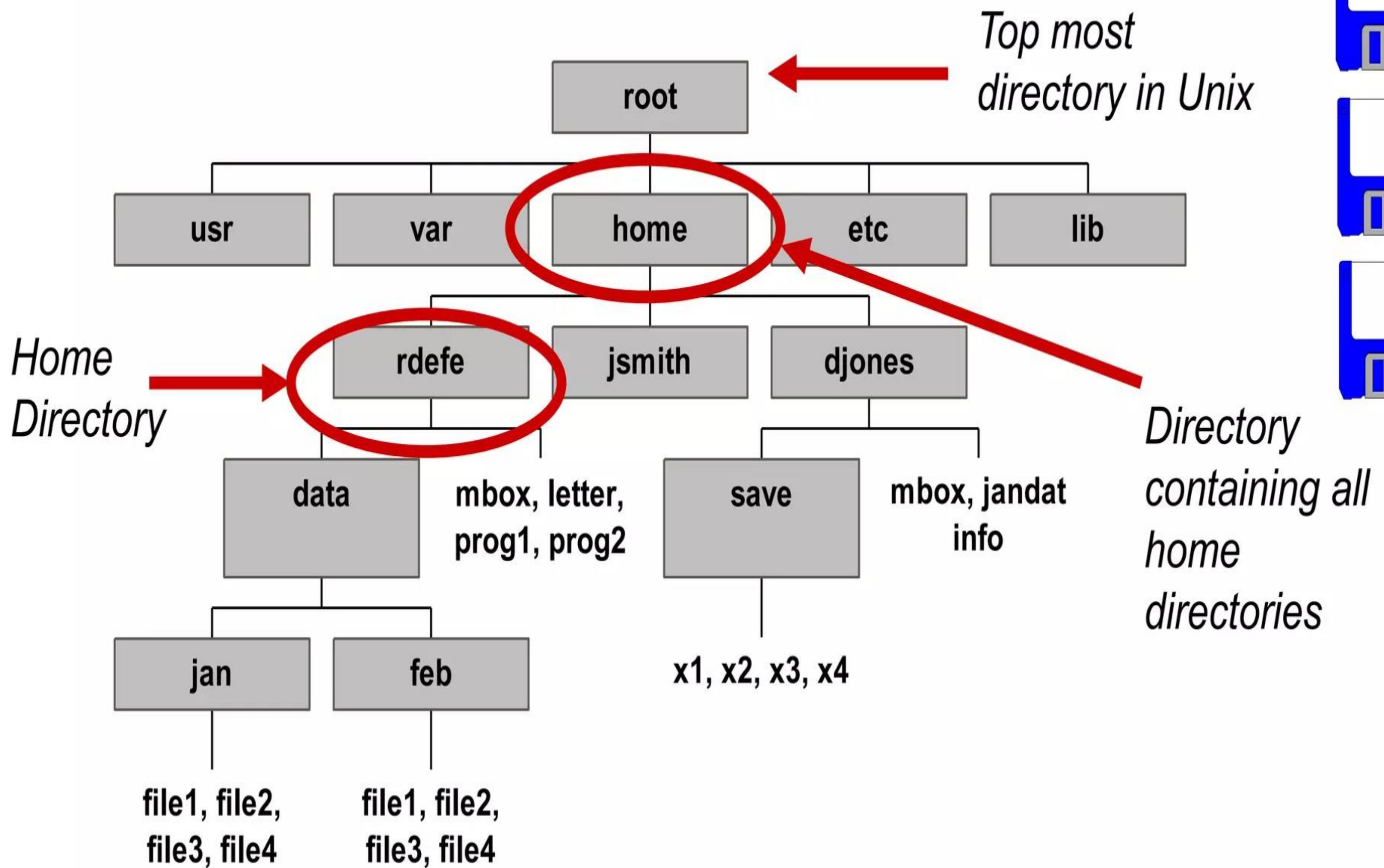


Figure 1-5 Layers of a UNIX system

Simple Unix Directory Structure



Unix File System

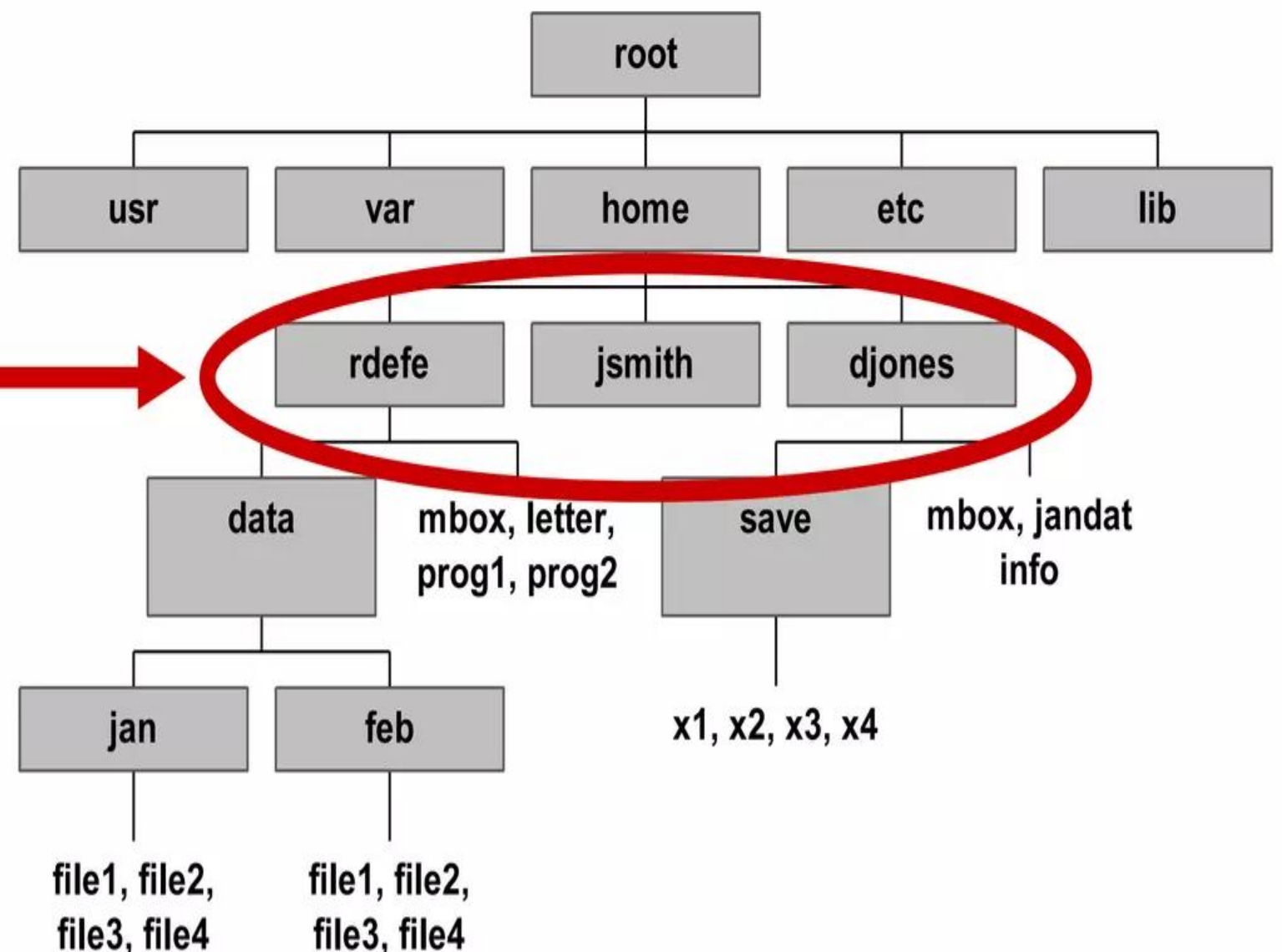


Unix File System

⌘ Home Directory

- ⊞ Personal Directory to store files
- ⊞ Usually the Directory Name = login ID
- ⊞ Current Directory after successful login

*Home directories for:
rdefe, jsmith and
djones*



Introducing UNIX Shells

A shell is a UNIX program that interprets the commands you enter from the keyboard

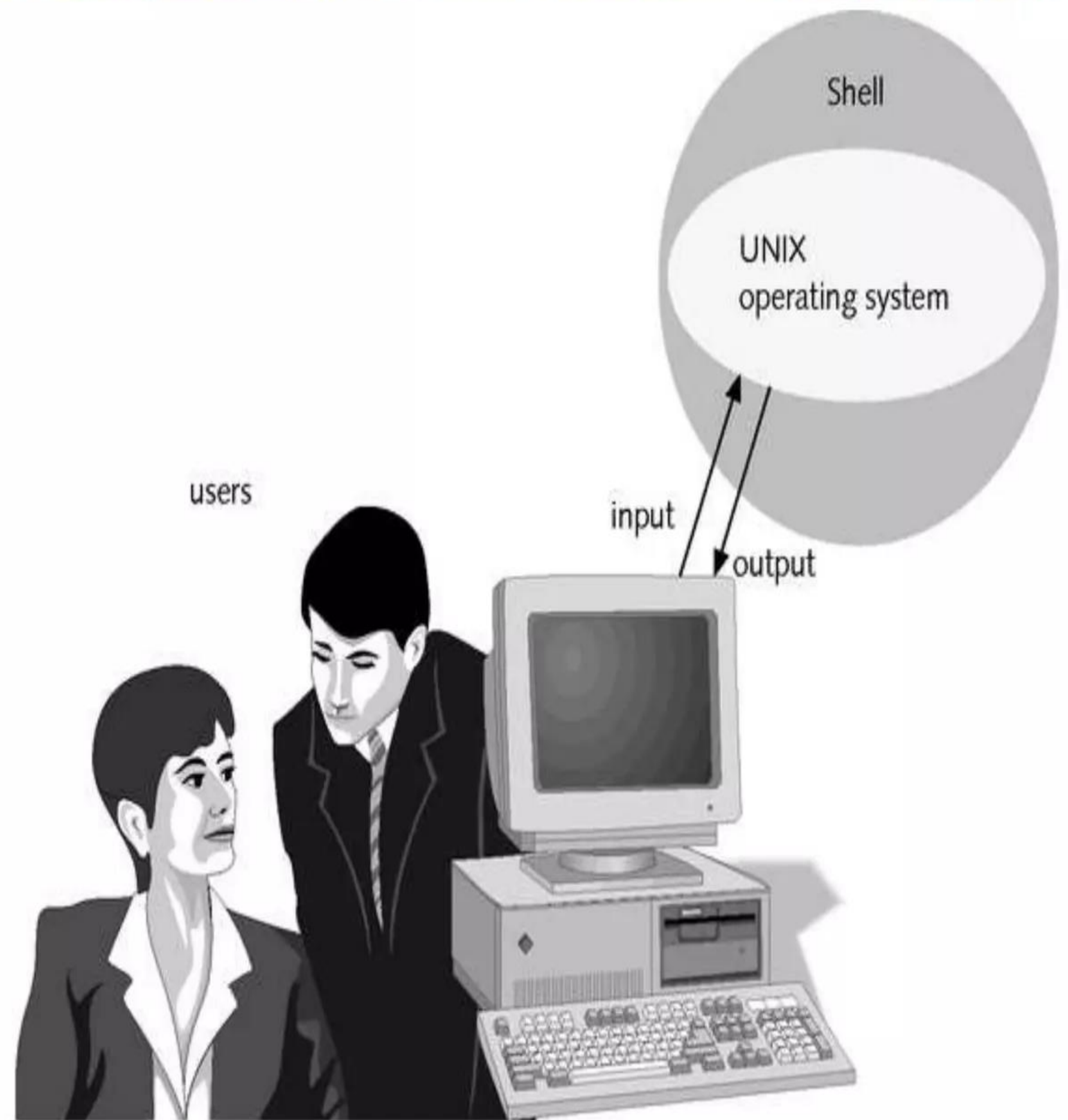


Figure 1-6 Shell's relationship to the user and the hardware

Choosing Your Shell

- ⌘ Shells interpret commands and act as first-class programming languages
- ⌘ A default shell is associated with your account when created – Bash is the default shell in Linux
- ⌘ A short list of some UNIX shells:
 - ☐ Bourne
 - ☐ Cshell
 - ☐ Korn
 - ☐ Bash

Logging in to UNIX



- ⌘ Log in by entering username and password when UNIX system booted or connected to
- ⌘ Enter at prompt (command-line mode) or into login box (GUI mode)
- ⌘ You're at the Shell prompt -- Now commands can be issued at the command prompt

Entering Commands



- ⌘ To interact with UNIX, a command is entered at the command prompt
- ⌘ UNIX is case-sensitive and most commands are typed in lower case
- ⌘ Two categories of commands
 - ☑ User-level: perform tasks
 - ☑ System administration: system management

Unix Commands

⌘ **passwd** : Change your password.

☑ This will let you enter a new password. Use a password that is not a real word or name and has numbers or punctuation in it.

⌘ **who**

☑ By typing “**who**” you will get a list with all the people logged in at that moment. The leftmost column shows the terminal at which the person is working, the next column shows the date and the rightmost column shows the computer number (IP number).

⌘ **who am i**

☑ This Check the current being user. It returns username that you are using.

Unix commands



Passwd command usage:

```
sariyer:~> passwd  
Changing password for dag.  
Old password:  
New passwd:  
Retype new passwd:  
sariyer:~>
```

Unix Commands

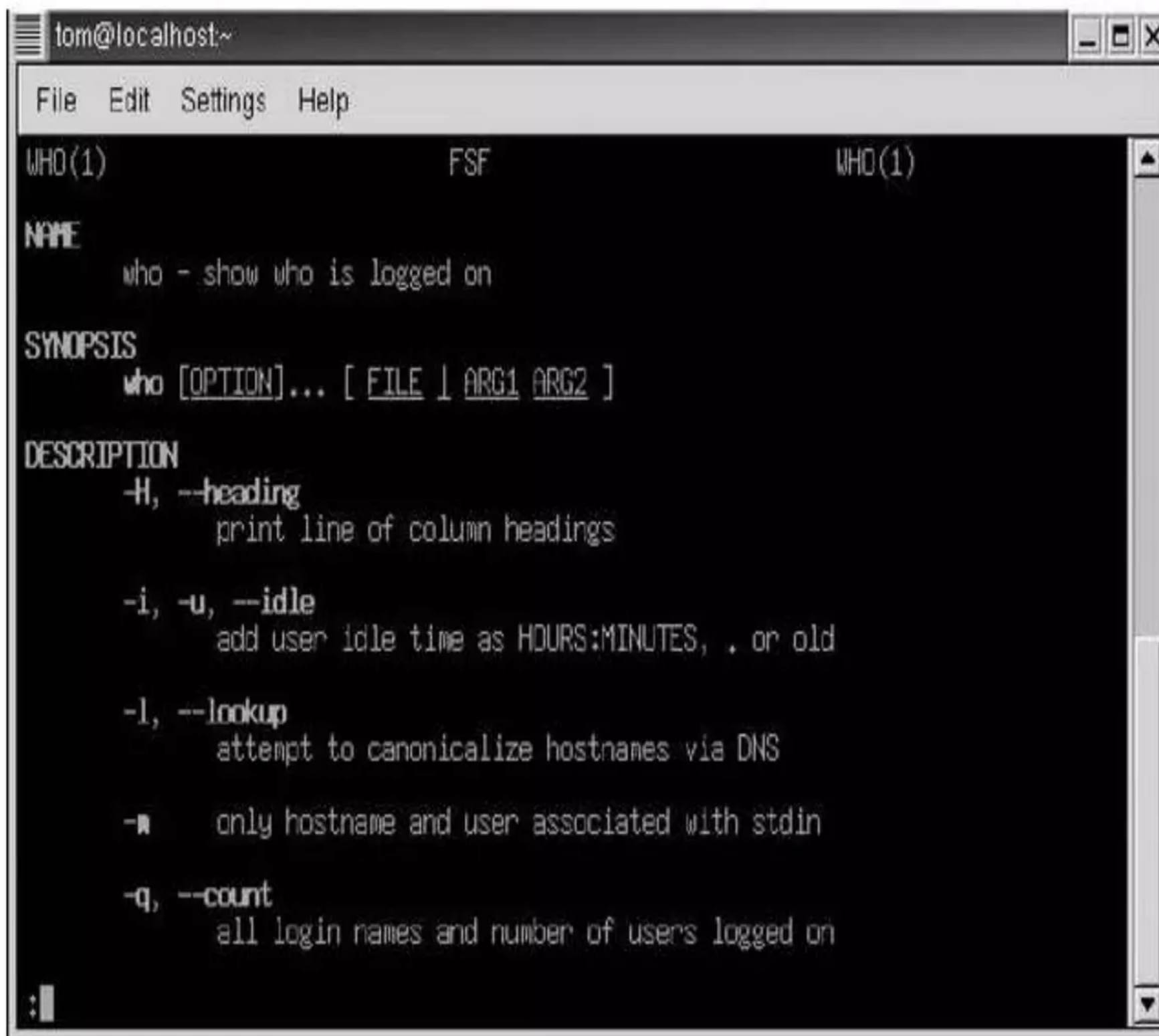
✂ man

- ☑ Show any UNIX command usages.
- ☑ “**man command**” shows purpose of command, its format, how to specify options and usage examples. We should use “**man**” to learn more about Unix commands given in this documents.
- ☑ Usage: #man who

✂ cal

- ☑ Shows calendar of the current month.
- ☑ Usage: #cal

Unix Commands



The screenshot shows a terminal window titled 'tom@localhost:~'. The window displays the manual page for the 'who' command. The page is divided into sections: NAME, SYNOPSIS, and DESCRIPTION. The NAME section shows 'who - show who is logged on'. The SYNOPSIS section shows 'who [OPTION]... [FILE | ARG1 ARG2]'. The DESCRIPTION section lists several options: -H, --heading (print line of column headings), -i, -u, --idle (add user idle time as HOURS:MINUTES, . or old), -l, --lookup (attempt to canonicalize hostnames via DNS), -n (only hostname and user associated with stdin), and -q, --count (all login names and number of users logged on).

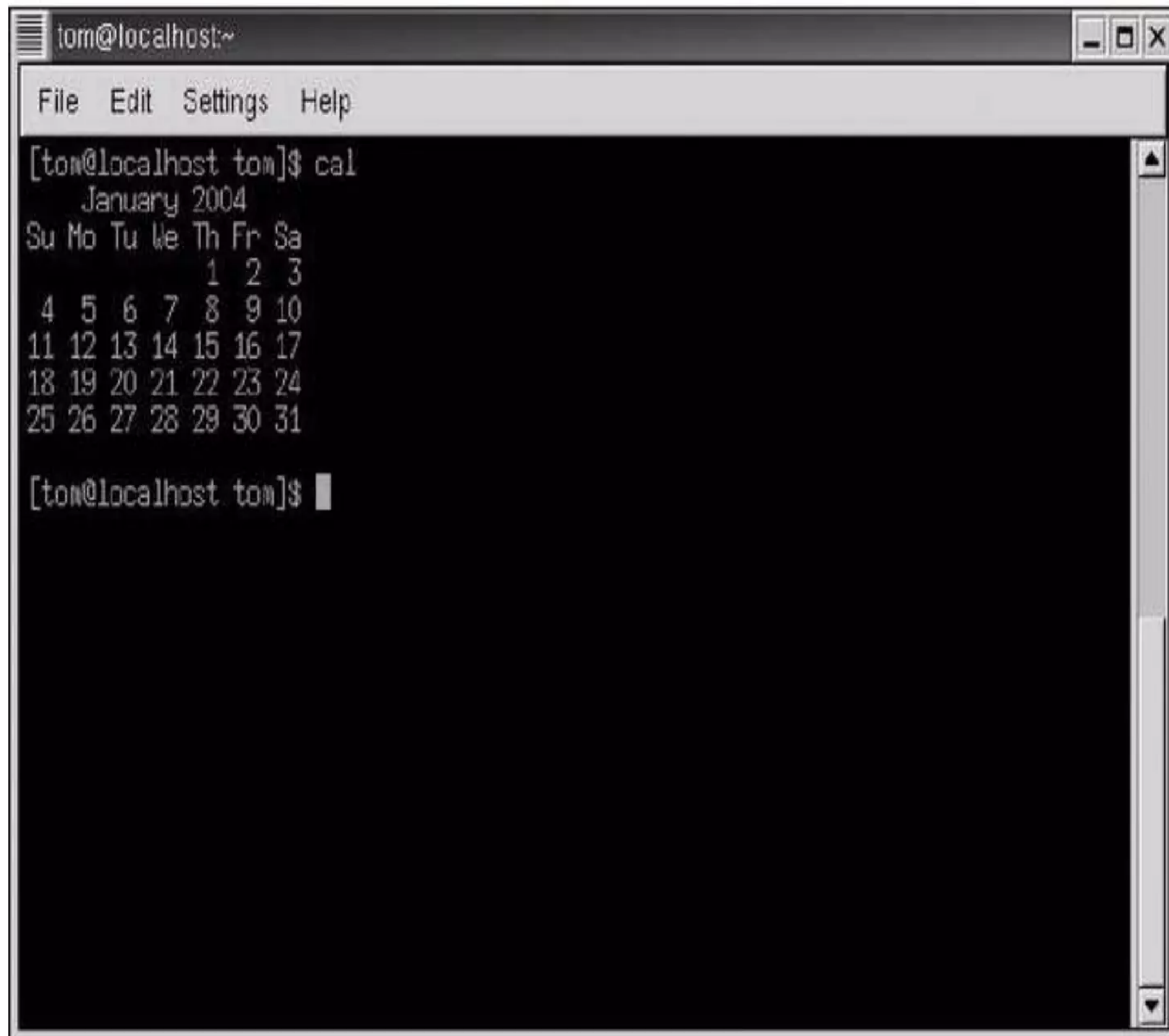
```
tom@localhost:~  
File Edit Settings Help  
WHO(1) FSF WHO(1)  
NAME  
  who - show who is logged on  
SYNOPSIS  
  who [OPTION]... [ FILE | ARG1 ARG2 ]  
DESCRIPTION  
  -H, --heading  
    print line of column headings  
  -i, -u, --idle  
    add user idle time as HOURS:MINUTES, . or old  
  -l, --lookup  
    attempt to canonicalize hostnames via DNS  
  -n  
    only hostname and user associated with stdin  
  -q, --count  
    all login names and number of users logged on  
:
```

The *man* program displays the UNIX online reference manual, called the man pages, for help purposes

Figure 1-11 Manual page

To exit
Press "q"

Unix Commands



The screenshot shows a terminal window titled 'tom@localhost:~'. The menu bar includes 'File', 'Edit', 'Settings', and 'Help'. The command prompt is '[tom@localhost tom]\$'. The user has entered the command 'cal', which displays the calendar for January 2004. The calendar is formatted as follows:

```
January 2004
Su Mo Tu We Th Fr Sa
          1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

The command prompt is now '[tom@localhost tom]\$' with a cursor.

Figure 1-8 Example of cal (current month) command

Unix Commands

⌘ ps

☑ Show process status.

☑ “ps” show processes own by your user. “ps -ax” shows all processes currently running on your server. The output is formatted in columns. First column is process ID, second column is process status and last column is command name.

Unix Commands



⌘ mkdir

- ☑ Create a new directory.
- ☑ “**mkdir dirname**” will create a new subdirectory called “**dirname**”.
- ☑ Usage: #mkdir test

⌘ rmdir

- ☑ Create a new directory.
- ☑ “**rmdir dirname**” will remove a subdirectory “**dirname**”.
The directory must be completely empty of files.
- ☑ Usage: #rmdir test

Unix Commands

⌘ cp

☑ Copy a file.

☑ “**cp src dest**” will make an exact copy of file “**src**”, with the name “**dest**”. If “**dest**” is a subdirectory name, the command will instead copy file “**src**” into the subdirectory “**dest**” and use its original file name.

⌘ mv

☑ Move (rename) a file.

☑ “**mv src dest**” will move file “**src**” to file “**dest**”. If “**dest**” is a subdirectory name, the command will instead move file into the subdirectory “**dest**” and use its original file name.

⌘ rm

☑ Remove (delete) a file..

☑ “**rm filename**” will delete “**filename**”. Once it is removed, there is no way to get it back!

Unix Commands

⌘ ls

☑ List Files & Directories



```
$ ls
198
A
aa.bb.cc.dd
bin
datefile
etc
mail
mbox
mbox.old
unix
$
```

Unix Commands

⌘ ls -p

▢-p *Show Directories*



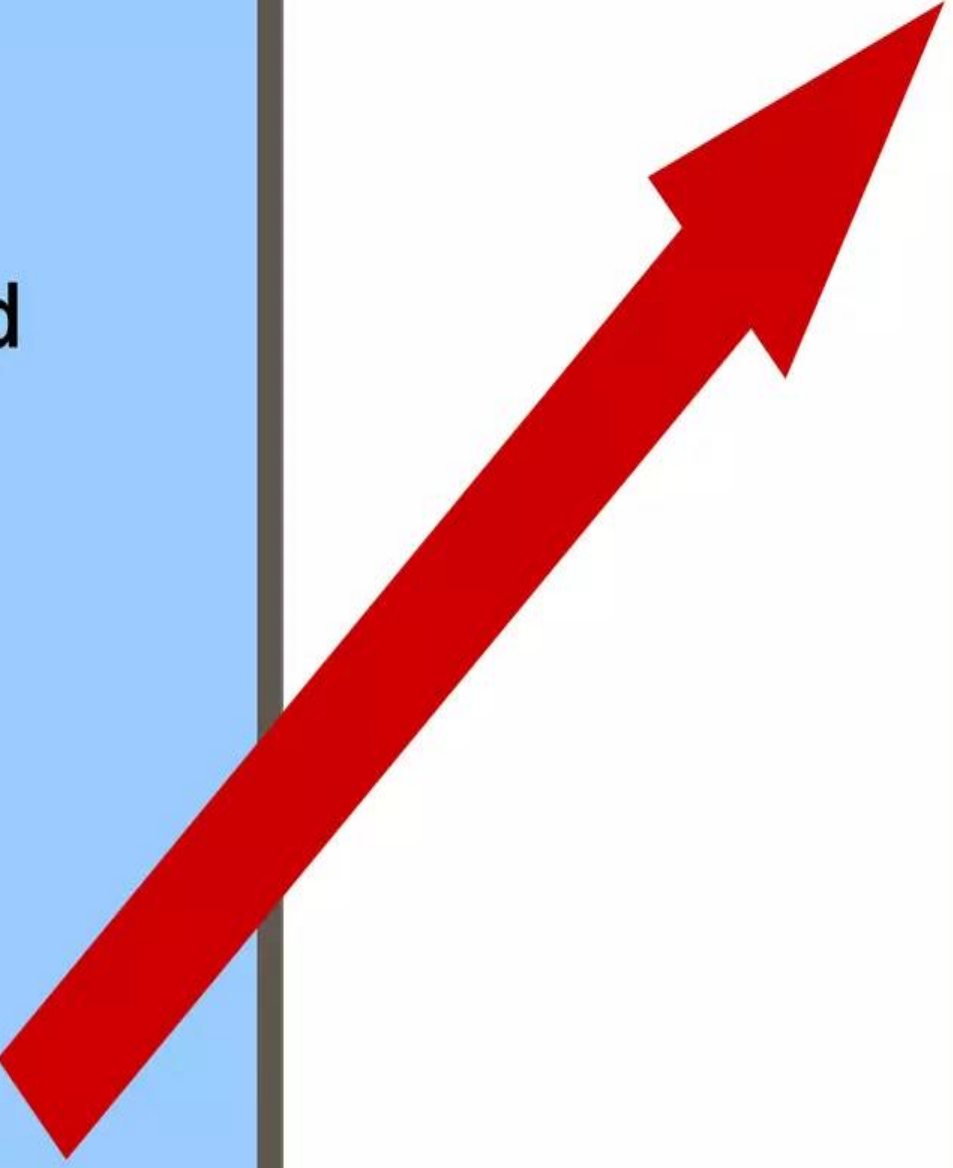
```
$ ls -p
198
A
aa.bb.cc.dd
bin/
datefile
etc/
mail/
mbox
mbox.old
unix/
$
```

Unix Commands

⌘ `ls [directory1] [directory2] ...`

☑ List the contents of a directory

```
$ ls -p
198
A
aa.bb.cc.dd
bin/
datefile
etc/
mail/
mbox
mboxold
unix/
$
```



```
$ ls unix
awk.data
awk.prg
bgraph
cpy
homework2
homework3
unix2test
$
```

Unix Commands

⌘ ls -s

▣ **-s** *Show File Size in Blocks*

▣ Block Sizes

▣ 512 bytes

▣ 1024 bytes

▣ 2048 bytes

```
$ ls -s
total 19
 1 198
 1 A
 1 aa.bb.cc.dd
 1 bin
 1 datefile
 1 etc
 1 mail
 2 mbox
 1 unix
```

Unix Commands

⌘ ls -l

📁 -l *Long Listing*

```
$ ls -l
total 19
-rw-r--r--  1 rdefe  unix  53 Sep 12 21:46 198
-rw-r--r--  1 rdefe  unix  62 Sep 12 21:47 A
drwxr-xr-x  2 rdefe  unix  96 Sep  4 20:58 bin
drwxr-xr-x  3 rdefe  unix  80 Aug 29 20:54 db
drwxr-xr-x  2 rdefe  unix  64 Aug 29 20:54 etc
drwxr-xr-x  2 rdefe  unix  32 Sep  2 1992 mail
-rw-----  1 rdefe  unix 984 Sep 12 21:44 mbox
drwxr-xr-x  2 rdefe  unix 144 Aug 29 20:54 unix
$
```

Unix Commands

⌘ ls -a

⬆ -a *List All Files*

*Include Files That
Begin with a '.'*

*Files that begin with a '.' are usually
configuration files*

```
$ ls -a
.
..
.profile
.sh_history
198
A
bin
datefile
db
etc
mbox
unix
$
```

Unix Commands

⌘ ls -r

▣ -r *Reverse Sort*

⌘ ls -R

▣ -R *List Contents of Subdirectories*

- *List files in each subdirectory.*
- *Subdirectories with Subdirectories will also be listed...and so on...*

```
$ ls -r
unix
mbox
mail
etc
db
datefile
bin
A
198
$
```

Unix Commands

⌘ `ls -l -t` *or* `ls -lt`

⏏ `-t` *Sort On Modification Time*

```
$ ls -lt
total 19
-rw-r--r--  1 rdefe  unix   62 Sep 12 21:47 A
-rw-r--r--  1 rdefe  unix   53 Sep 12 21:46 198
-rw-----  1 rdefe  unix  984 Sep 12 21:44 mbox
drwxr-xr-x  2 rdefe  unix   96 Sep  4 20:58 bin
drwxr-xr-x  2 rdefe  unix   64 Aug 29 20:54 etc
drwxr-xr-x  2 rdefe  unix  144 Aug 29 20:54 unix
drwxr-xr-x  3 rdefe  unix   80 Aug 29 20:54 db
drwxr-xr-x  2 rdefe  unix   32 Sep  2  1992 mail
$
```

Unix Commands

⌘ `ls -l -t -r` *or* `ls -ltr`

⏮ **-tr** *Reverse Sort On Modification Time*

```
$ ls -ltr
total 19
drwxr-xr-x  2 rdefe  unix   32 Sep  2  1992 mail
drwxr-xr-x  3 rdefe  unix   80 Aug 29 20:54 db
drwxr-xr-x  2 rdefe  unix  144 Aug 29 20:54 unix
drwxr-xr-x  2 rdefe  unix   64 Aug 29 20:54 etc
drwxr-xr-x  2 rdefe  unix   96 Sep  4 20:58 bin
-rw-----  1 rdefe  unix  984 Sep 12 21:44 mbox
-rw-r--r--  1 rdefe  unix   53 Sep 12 21:46 198
-rw-r--r--  1 rdefe  unix   62 Sep 12 21:47 A
$
```

Unix Commands

⌘ cat [Filename₁] [Filename₂] ...

☑ Concatenate Files

☑ Display Files to the Screen

☑ Ctrl-s *Pause Screen*

☑ Ctrl-q *Unpause Screen*

```
$ cat sample.file
```

```
This is a sample file that i'll use to demo how  
the pr command is used.
```

```
The pr command is useful in formatting  
various types of text files.
```

```
$
```

Unix commands

cat command usage:

Prompt

Command

```
bagriy@sariyer:~> cat
Help! I'm stuck in a Linux program!
Help! I'm stuck in a Linux program!
```

If you type this row and then press enter

The text indicates what we typed to cat

Unix Commands

⌘ `more [Filename1] [Filename2] ...`

⏏ Display Files to the screen one page at a time

```
$ more largefile
This is a sample file that i'll use to demo how
the pr command is used.
The pr command is useful in formatting
various types of text files.
```

--More-- (16%)

⌘ **Spacebar** (next page)

⌘ **b** (previous page)

⌘ **Enter Key** (next line)

⌘ **q** (Quit & return to unix)

⌘ **/string** (search for string)

⌘ **:n** (next file)

⌘ **:p** (previous file)

Unix Commands

⌘ pr [Filename₁] [Filename₂] ...

☑ Format & Display Files to the Screen

```
$ pr sample.file
```

```
98-05-22 23:07          sample.file          Page 1
```

```
This is a sample file that i'll use to demo how  
the pr command is used.
```

```
The pr command is useful in formatting  
various types of text files.
```

```
$
```

Unix Commands

⌘ pr -[dn] [Filename₁] [Filename₂] ...

⏏ -d *Double Space Output*

⏏ -n *Number Lines*

```
$ pr -d sample.file
```

```
98-05-22 23:07    sample.file    Page 1
```

```
This is a samp
```

```
the pr command
```

```
The pr command
```

```
various types
```

```
$
```

```
$ pr -n sample.file
```

```
98-05-22 23:07    sample.file    Page 1
```

```
1  This is a sample file that i'll use to demo how the
2  pr command is used.
3  The pr command is useful in formatting
4  various types of text files.
```

```
$
```

Unix Commands



⌘ grep

☑ Search file and print lines that match pattern.

☑ “**grep pattern filename**” will print out each line in file “**filename**” that contains “**pattern**”. It is case-sensitive search.

Unix Commands

⌘ `lpr [Filename1] [Filename2] ...`

☑ Send Files to a Unix Printer

☑ No Formatting (page numbers, etc.)

```
$ lpr prog1 mbox
$
```



*Ideal solution would
be to combine the
lpr & pr to achieve
formatted printed
output*



```
$ pr sample.file
```

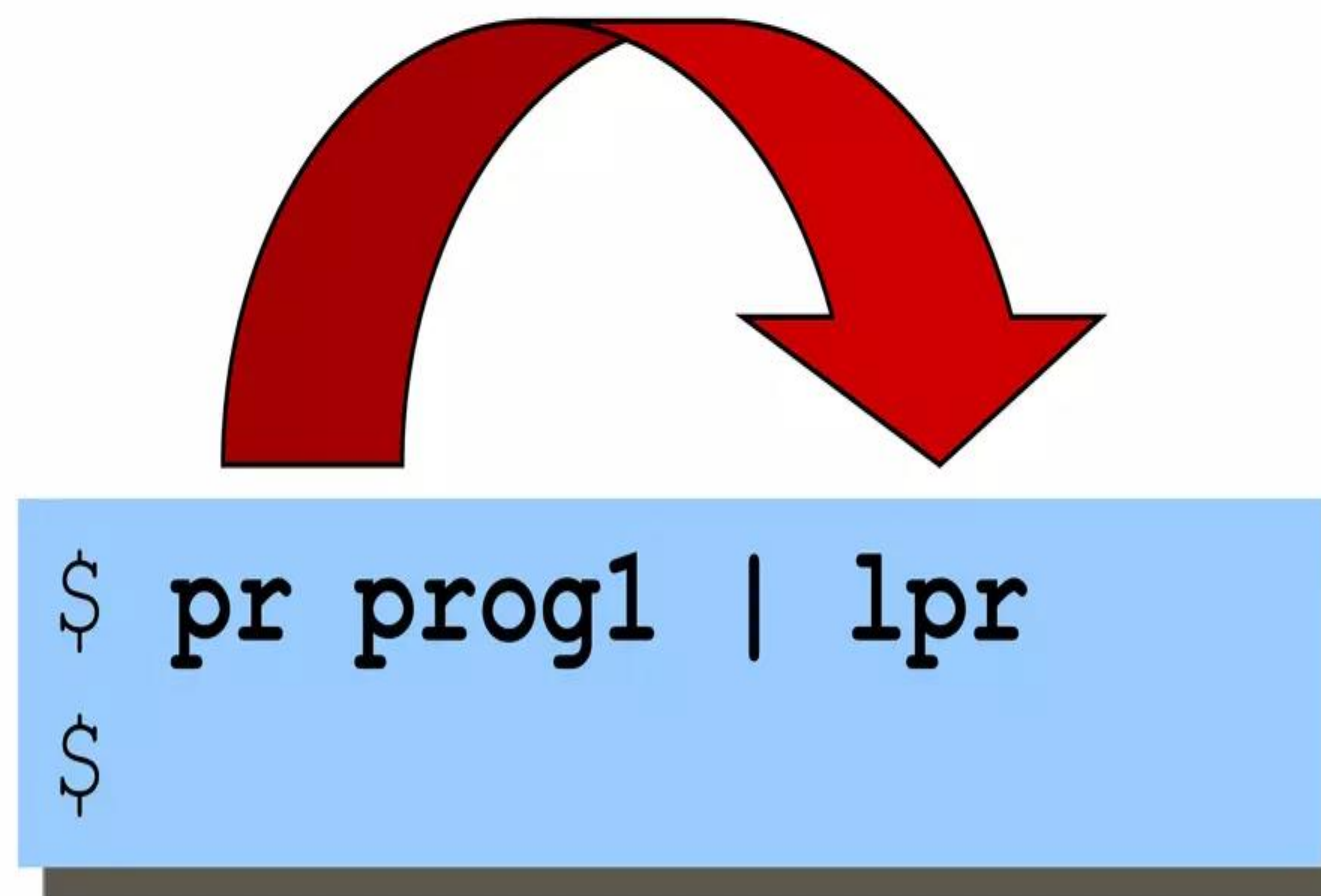
```
98-05-22 23:07 sample.file Page 1
```

```
This is a sample file that i'll use  
to demo how the pr command is used.
```

```
...
```

Unix Redirection

⌘ Output Normally Displayed to the Screen is Redirected and Becomes the Input to Another Command

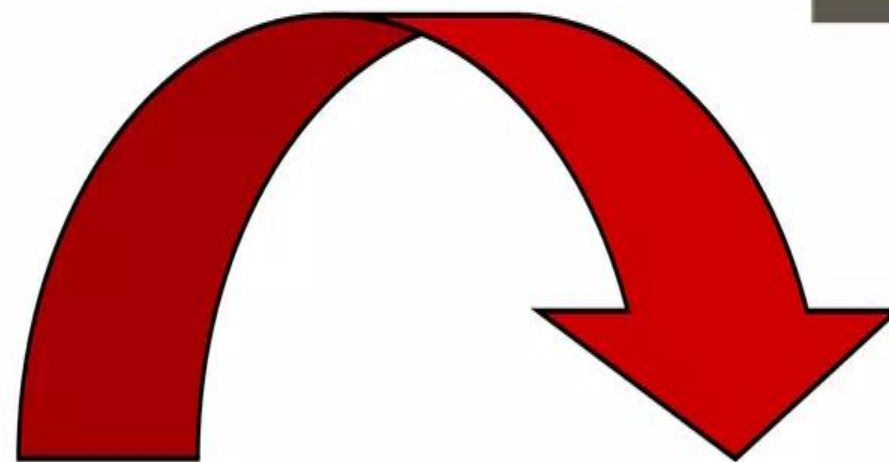


Unix Redirection

⌘ The Output of One Command can be used as the Input to Another

*Command that
generates screen
output*

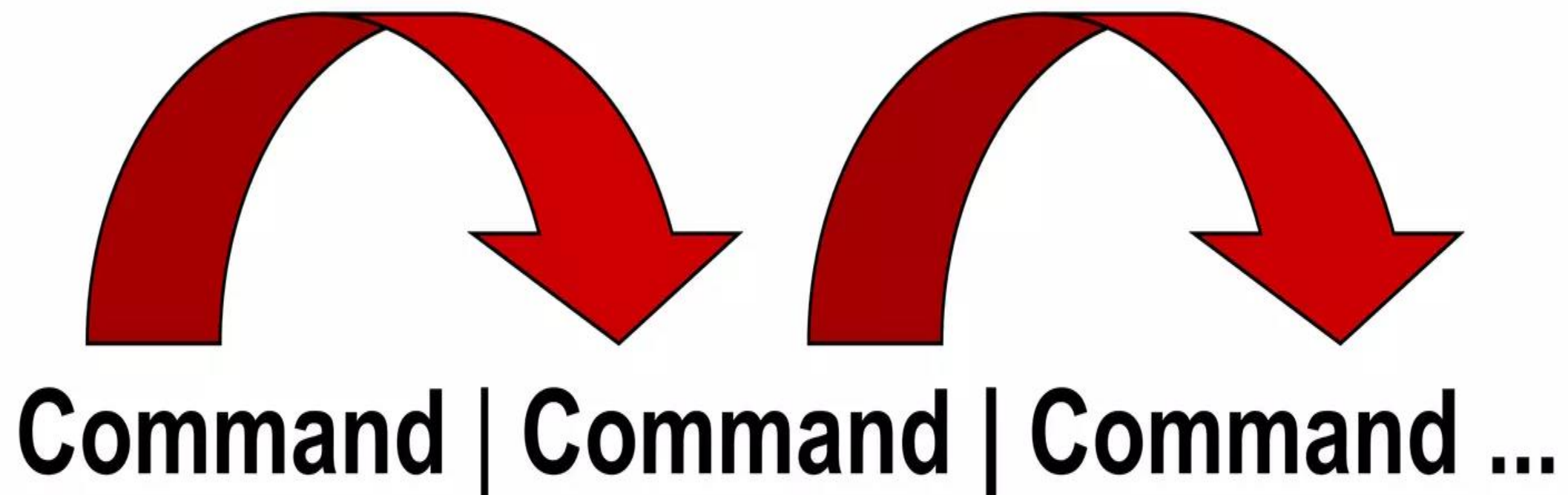
```
$ pr prog1 | lpr  
$
```



Command | Command

*Command that
requires **input***

Unix Redirection



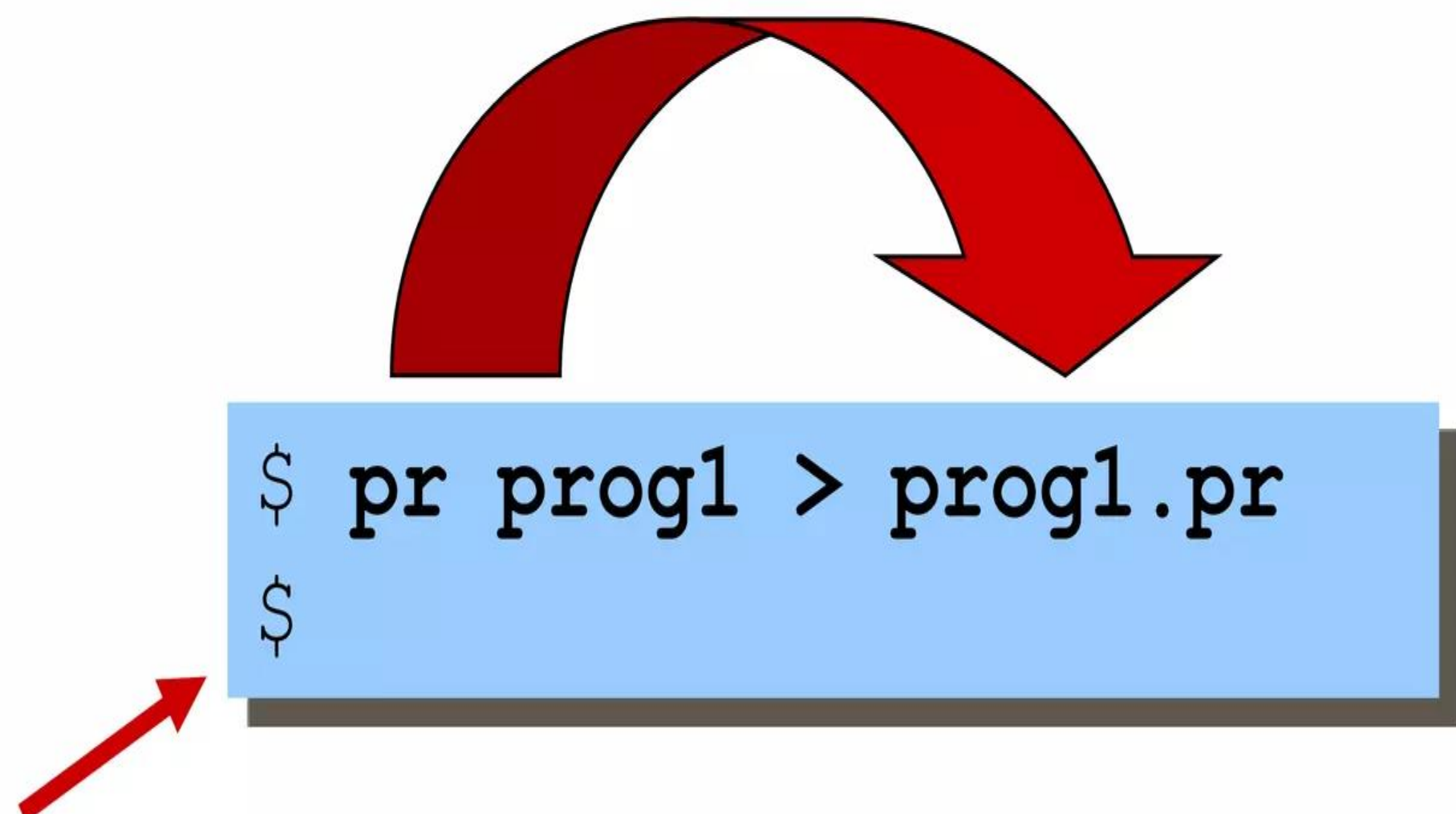
```
$ ls | pr | lpr
```

```
$
```

*Directory Listing Formatted
Using pr, Printed on Paper*

Unix Redirection

⌘ Output Normally Displayed to the Screen
Can Be Redirected to a File



```
$ pr prog1 > prog1.pr  
$
```

Note: Nothing is displayed to the screen

The file **prog1.pr** contains the output of the *pr* command

Unix Redirection

⌘ Redirecting Output

▣ The Output of Command can be saved to a File

*Command that
generates screen
output*

```
$ pr prog1 > prog1.pr  
$
```

Command > File

*File used to capture
(save) screen output*

Caution: Output File will
be erased if it already
exists.

Redirection Examples

1

```
$ pr data > data.pr  
$ lpr data.pr
```

Is the same as

```
$ pr data | lpr
```

2

```
$ cal 3 1993 > year
```

Using arguments with redirection

3

```
$ ls -l data > stuff
```

Using arguments & options with redirection

Redirection Examples

4 `$ pr data > lpr`

5 `$ cat text let > mesg`

6 `$ pr f1 f2 > f1`

1

*Create the file **f1**
Erases **f1** if it already
exists*

2

*pr **f1** and **f2**, redirect to **f1***

Unix Commands

⌘ head

☐ “head” will display the first ten lines in the listed files.

☐ `head [- lines] [file1 file2 ... fileN]`

☐ Any numeric option will be taken as the number of lines to print, so `head -15 frog` will print the first fifteen lines of the file `frog`.

Unix Commands

⌘ tail

▣ Like **head**, **tail** display the last ten lines in the listed files.

▣ **tail** [- *lines*] [*file1 file2 ... fileN*]

▣ Any numeric option will be taken as the number of lines to print, so `tail -15 frog` will print the last fifteen lines of the file `frog`.

Unix Commands

⌘ **wc**

☑ **wc** (word count) simply counts the number of words, lines, and characters in the file(s).

☑ **wc [-clw] [*file1 file2 ... fileN*]**

☑ The three parameters, **clw**, stand for character, line, and word respectively, and tell **wc** which of the three to count.

Unix Commands



⌘ **cmp**

▣ **cmp** compares two files.

▣ **cmp** is very simple, and merely tells you where the two files first differ.

▣ **cmp** file1 [*file2*]

Unix Commands



⌘ vi [Filename]

- ☑ Full Screen, Visual Editor
- ☑ Changes Displayed As They Are Made
- ☑ Edits A Copy of the File
- ☑ Changes Must Be Saved

```
$ vi msg
```

Unix Commands

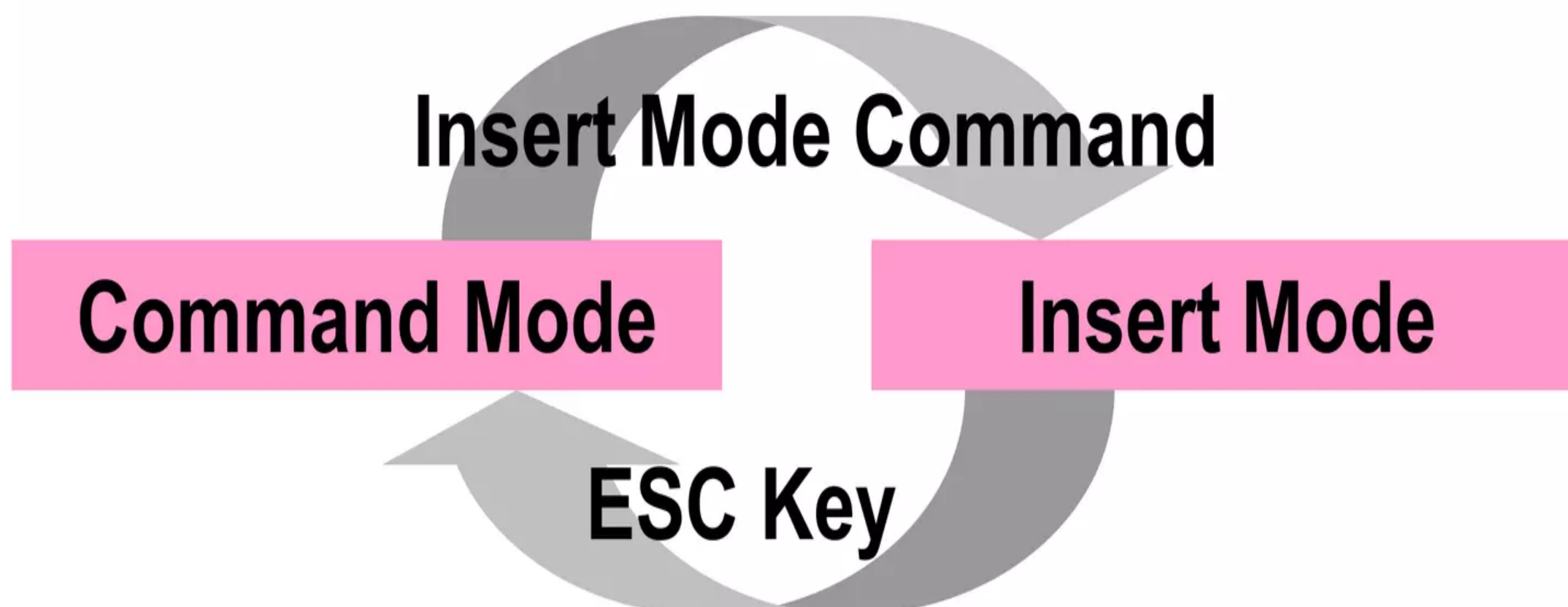
Vi Modes

⌘ Command Mode

- ▣ Start in command mode
- ▣ Every Key Performs an Editing Command

⌘ Insert Mode

- ▣ Perform Text Entry/Input



Sample vi Edit Session

```
$ vi msg
```



```
Welcome to UNIX
```

```
This
```

```
a te
```

```
~
```

```
~
```

```
~
```

```
~
```

```
Wnewtextelcome to UNIX
```

```
This is an example of  
a text file in vi
```

```
~
```

```
~
```

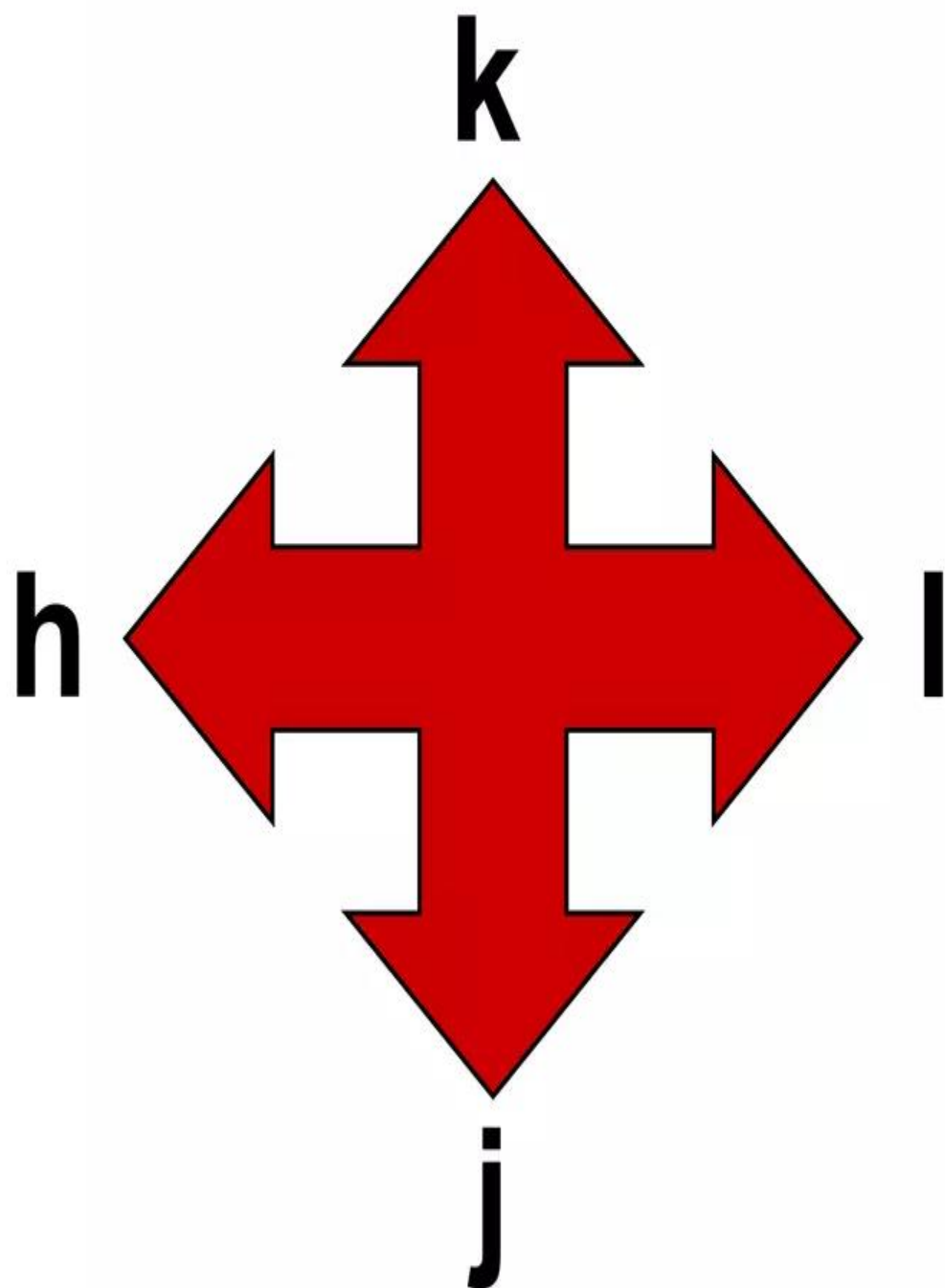
```
:wq
```

```
$
```

- ⌘ Start in Command Mode
- ⌘ Press **a** key for *Append After Cursor Command*
- ⌘ Enter text (Input Mode)
- ⌘ Press **ESC** key (Command Mode)
- ⌘ Press **:wq** followed by Enter key to *Save & Exit*

Cursor Positioning

⌘ Moving the Cursor in Command Mode



Key	Command
w	<i>Forward One Word</i>
b	<i>Backward One Word</i>
Cntrl-d	<i>Scroll Down</i>
Cntrl-u	<i>Scroll Up</i>
Enter	<i>Down One Line</i>
G	<i>Go To Last Line</i>
5G	<i>Go To Line 5</i>

Text Input Mode

vi Text Input Mode Commands

Wnewtextelcome to UNIX

a *Append After Cursor*

Welcome to UNIXnewtext

A *Append at End of Line*

Th newtextWelcome to UNIX

i *Insert Before Cursor*

Th newtextWelcome to UNIX

I *Insert at Beginning of Line*

Th newtext
Welcome to UNIX

O *Open New Line Above Cursor*

o *Open New Line Below Cursor*

This is an example of
a text file in vi

R *Replace Text (typeover)*

r *Replace One Char*

Deleting Text

Welcome to UNIX

This is an example of

a t

~

~

This is an example of
a text file in vi

~

~

x *Delete One Char*

X *Delete Char Before Cursor*

dd *Delete One Line*

⌘ Proceed Any Command with a Number to multiply effect

⌘ 4x *Delete 4 chars*

⌘ 5dd *Delete 5 lines*

⌘ 3dw *Delete 3 words*

Undo Changes

x *Delete One Char*

u *Undo Last Change*

U *Undo All Changes on Line*

Welcome to UIX

Th Welcome to UNIX

a

~ This is an example of

~ a te WELcome to UIX 123

~

~ This Welcome to UNIX

a tex

~

~ This is an example of
a text file in vi

~

~

Saving Your Changes



⌘ Command Mode

⌘ **ZZ**

Save & Exit vi

⌘ **:wq**

Save & Exit vi

⌘ **:w data**

Save to a file data

⌘ **:q!**

Quit Without Saving Changes